

Principia Render Graph, Shader Tooling, and Quadtree Design

Implementation-oriented design note

Scope. This document writes up the proposed rendering architecture discussed for Principia: a composable colour/lightness pipeline, hot-reloadable shader tooling, built-in and custom shader nodes, debug visualisation infrastructure, and a Google-Maps-style quadtree tile system with ancestor fallback, panning, zooming, and adaptive refinement.

March 19, 2026

Contents

1	Design goals	2
2	High-level render architecture	2
3	Render contracts	3
3.1	Stable simulation-facing payload	3
3.2	Stage outputs	3
3.3	Why not force everything to be RGB?	4
4	Shader node model	4
4.1	Node kinds	4
4.2	Suggested TypeScript model	4
5	Helper library for shaders	5
5.1	Colour-space helpers	5
5.2	Scalar and transfer-function helpers	5
5.3	Shape-sphere helpers	5
5.4	Shape-sphere colour helpers	5
5.5	Why this matters	6
6	Hot reloading and live shader editing	6
6.1	Compilation flow	6
6.2	Uniform exposure	6
6.3	Failure handling	7
7	UI model	7
7.1	Main structure	7
7.2	Progressive disclosure	7
8	Debug shaders as first-class infrastructure	8
8.1	Sample-level debug views	8
8.2	Tile-level debug views	8
8.3	Overlay-style structural debug views	9
8.4	Recommended enum model	9
9	Google-Maps-style quadtree model	9
9.1	Tile pyramid	9
9.2	Camera in world coordinates	9
9.3	Base zoom level	10
9.4	Visible tile range	10
9.5	Ancestor fallback	10
9.6	Render strategy: active tile quads	10
10	Quadtree scheduler and refinement	11
10.1	Tile lifecycle	11
10.2	Pan and zoom behaviour	11
10.3	Refinement priority	11
11	Precision strategy for deep zoom	11
12	Recommended implementation order	12

13 Suggested project structure	13
14 Summary	13

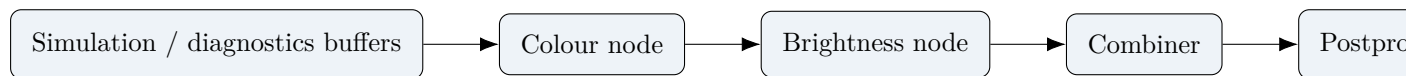
1 Design goals

The proposed system should satisfy five requirements.

1. **One canonical simulation payload.** The expensive compute stage should produce stable summaries that can drive rendering, debugging, refinement, and export.
2. **Composable render logic.** Colour and brightness should be selected independently instead of being hard-coded into a flat list of monolithic render modes.
3. **Shared tooling for built-ins and user shaders.** The same helper library, data contracts, and hot-reload path should be used both internally and for advanced user customisation.
4. **First-class debugging.** Tile state, cache state, refinement priority, sample flags, and quadtree boundaries should all be visualisable through the same render graph.
5. **Slippy-map interaction model.** Panning and zooming should behave like a tiled map viewer: visible tiles appear immediately from the nearest ready ancestor, then sharpen progressively as higher resolution tiles finish.

2 High-level render architecture

The core idea is to split the rendering system into fixed stages with clear contracts, rather than a single giant mode switch.



A cleaner conceptual version is:

```
sim payload
-> colour encoder
-> lightness / brightness encoder
-> combiner
-> postprocess / overlays
-> screen
```

This keeps the scientific semantics legible:

- the colour stage encodes category, geometric position on the shape sphere, or some diagnostic code,
- the brightness stage encodes a scalar field such as event time, diffusion, FTLE, or ensemble spread,
- the combiner turns those into final display colour,
- postprocess handles things like colour-vision deficiency previews, inversion, or debug overlays.

3 Render contracts

3.1 Stable simulation-facing payload

Custom shaders should not reach directly into arbitrary internal buffers. Instead, the compute stage should expose a stable render-facing contract built from the sample and tile summaries already defined by the simulator.

A practical structure is:

```
struct RenderSample {
    event_code: u32,
    outcome_class: u32,
    status_flags: u32,

    event_time: f32,
    diffusion: f32,
    ftle: f32,
    ensemble_spread: f32,
    coherence: f32,
    energy_drift: f32,
    d_min: f32,
    arc_length_n: f32,

    n_repr: vec3f,
    n_mean: vec3f,
    checkpoint_count: u32,
    checkpoints: array<vec4f, M> // xyz = n(t), w = unwrapped phase
}

struct RenderTile {
    tile_depth: u32,
    tile_state: u32,
    coherence_score: f32,
    outcome_impurity: f32,
    ensemble_spread: f32,
    suspect_fraction: f32,
    priority_score: f32,
    ancestor_gap: u32,
    cache_age: f32
}

struct RenderContext {
    sample: RenderSample,
    tile: RenderTile,
    uv: vec2f,
    screen_uv: vec2f,
    time: f32
}
```

The exact field list can change, but the main rule should remain: *render code sees a stable logical context, not raw storage layout details.*

3.2 Stage outputs

A pragmatic public contract is:

```
colour(ctx)      -> vec3    // normalized linear RGB
brightness(ctx) -> float    // nominally in [0,1]
```

```
combine(rgb,b)  -> vec3
post(rgb)       -> vec3
```

This keeps custom node authoring straightforward. Internally, built-in shaders are free to work in OKLab or OKLCH before returning linear RGB.

3.3 Why not force everything to be RGB?

For the public node interface, RGB plus scalar brightness is simple and usable. Internally, however, the default combiner should still respect the conceptual split between chromatic content and lightness. In particular, the default pipeline can:

1. convert the colour node output from linear RGB to OKLab,
2. replace or modulate the lightness channel using the brightness scalar,
3. convert back to linear RGB,
4. apply gamut handling and optional postprocess passes.

That gives a principled default without making the public shader API awkward.

4 Shader node model

4.1 Node kinds

The render graph should treat built-ins and customs uniformly. At minimum there are four node kinds:

1. **Colour nodes** - event classification, shape sphere, diagnostic palettes, custom colour shaders.
2. **Brightness nodes** - fixed, event time, diffusion, FTLE, ensemble spread, custom brightness shaders.
3. **Combiner nodes** - initially a small built-in set, with a single default combiner as the main path.
4. **Postprocess nodes** - CVD preview, invert, exposure, debug compositing, overlays.

4.2 Suggested TypeScript model

```
type ShaderNodeKind = "colour" | "brightness" | "combiner" | "post";

type UniformField =
  | { kind: "float"; key: string; default: number; min?: number; max?: number; step?: number }
  | { kind: "int"; key: string; default: number; min?: number; max?: number; step?: number }
  | { kind: "bool"; key: string; default: boolean }
  | { kind: "vec3"; key: string; default: [number, number, number] }
  | { kind: "enum"; key: string; options: string[]; default: string }
  | { kind: "image"; key: string };

type ShaderNodeDef = {
  id: string,
  kind: ShaderNodeKind,
```

```
label: string,  
source: string,  
entryPoint: string,  
uniformsSchema: UniformField[],  
builtIn: boolean  
};
```

This schema should drive both the UI and the uniform packing logic.

5 Helper library for shaders

A proper shader SDK is worth adding immediately. It makes built-ins easier to write, makes custom shader authoring far more pleasant, and encourages reuse instead of everyone rewriting the same colour conversions and transfer functions.

5.1 Colour-space helpers

```
fn srgb_to_linear(c: vec3f) -> vec3f  
fn linear_to_srgb(c: vec3f) -> vec3f  
fn linear_rgb_to_oklab(c: vec3f) -> vec3f  
fn oklab_to_linear_rgb(lab: vec3f) -> vec3f  
fn oklab_to_oklch(lab: vec3f) -> vec3f  
fn oklch_to_oklab(lch: vec3f) -> vec3f  
fn clamp01(x: f32) -> f32  
fn saturate3(c: vec3f) -> vec3f
```

5.2 Scalar and transfer-function helpers

```
fn remap(x: f32, a: f32, b: f32, c: f32, d: f32) -> f32  
fn smooth_remap(x: f32, a: f32, b: f32) -> f32  
fn log_remap(x: f32, eps: f32) -> f32  
fn percentile_curve(x: f32, p0: f32, p1: f32) -> f32  
fn transfer_viridis(x: f32) -> vec3f  
fn transfer_cubehelix(x: f32) -> vec3f
```

5.3 Shape-sphere helpers

```
fn n_at(ctx: RenderContext, i: u32) -> vec3f  
fn phase_at(ctx: RenderContext, i: u32) -> f32  
fn final_n(ctx: RenderContext) -> vec3f  
fn checkpoint_mean_n(ctx: RenderContext) -> vec3f  
fn binary_collision_proximity(n: vec3f) -> f32  
fn lagrange_proximity(n: vec3f) -> f32
```

5.4 Shape-sphere colour helpers

```
fn vmf_oklab_6pole(n: vec3f, L: f32, C: f32, kappa: f32) -> vec3f  
fn vmf_okabe_ito(n: vec3f, L: f32, C: f32, kappa: f32) -> vec3f  
fn lut_sphere(n: vec3f, lut_id: u32, kappa: f32) -> vec3f  
fn direction_cosines_rgb(n: vec3f) -> vec3f
```

5.5 Why this matters

These helpers do three jobs:

- they reduce friction for custom shader authors,
- they let built-ins and customs share the same conceptual toolkit,
- they keep your own authoring workflow identical to the one exposed to advanced users.

6 Hot reloading and live shader editing

Hot reload should operate at the node-module level, while still rebuilding a complete fragment module under the hood.

6.1 Compilation flow

When a node changes:

1. hash the changed node source,
2. regenerate the full fragment source from the shared prelude, the selected node sources, and a fixed wrapper `main()`,
3. compile the shader module,
4. validate entry points and binding layout,
5. if successful, swap the pipeline,
6. if compilation fails, keep the last valid pipeline active and display errors inline.

A short debounce window, for example 100–200 ms after typing pauses, keeps the editor responsive without thrashing the compiler.

6.2 Uniform exposure

Uniforms should be schema-driven rather than guessed from raw WGSL. A node definition should declare its metadata separately, and the editor can auto-generate controls from that schema.

```
{
  "id": "shapeSphereCustom01",
  "kind": "colour",
  "entry": "colour_main",
  "imports": ["colour_spaces", "shape_sphere", "vmf"],
  "uniforms": [
    {"name": "kappa", "type": "f32", "default": 3.0, "min": 0.1, "max": 12.0},
    {"name": "chroma", "type": "f32", "default": 0.12, "min": 0.0, "max": 0.3}
  ]
}
```

6.3 Failure handling

Invalid custom shaders should be allowed to produce ugly output, but not to crash the application. The editor should therefore:

- keep the last valid version active,
- show compile errors with line and column information,
- highlight the relevant source region,
- mark the node as invalid until fixed.

7 UI model

The UI should mirror the render graph rather than a giant flat mode list.

7.1 Main structure

```
RENDER

COLOUR
  None
  Event Classification
  Shape Sphere
  Debug
  Custom

BRIGHTNESS
  Fixed
  Event Time
  Diffusion
  FTLE
  Ensemble Spread
  Debug
  Custom

COMBINER
  Principia Default
  Multiply
  Replace Lightness

POST
  None
  CVD Preview
  Invert
  Overlay
```

7.2 Progressive disclosure

When **Shape Sphere** is selected, reveal a nested panel with:

- sensible preset maps,
- palette family,
- chroma,

- κ ,
- seam / pole handling,
- image or URL-driven map source,
- deeper customisation controls.

When `Custom` is selected under colour or brightness, reveal the live shader editor, compile status, exposed uniforms, helper docs, and reset / duplicate actions.

8 Debug shaders as first-class infrastructure

Debug shaders should be part of the core design, not an afterthought. They will be essential for validating the quadtree, the cache, the tile scheduler, and the sample summaries long before the fully adaptive system is stable.

8.1 Sample-level debug views

Useful sample debug outputs include:

- event code or outcome class,
- suspect flags,
- energy drift,
- decode mode,
- event checkpoint index,
- encounter count or other compact diagnostics.

8.2 Tile-level debug views

The minimum tile debug set should include:

- tile depth,
- tile state,
- coherence score,
- outcome impurity,
- ensemble spread,
- suspect fraction,
- refinement priority score,
- cache age,
- ancestor gap for fallback rendering.

8.3 Overlay-style structural debug views

These should include:

- quadtree boundaries,
- active leaf outlines,
- fallback ancestor tinting,
- pending tile hatching,
- viewport-visible tile set,
- locked or stale tile markers.

8.4 Recommended enum model

```
enum TileState {
    Unseen,
    Queued,
    Computing,
    Ready,
    Stale,
    Evicted
}

enum RenderRole {
    Hidden,
    VisibleReady,
    VisibleFallback
}
```

9 Google-Maps-style quadtree model

The interaction model should follow the standard sloppy-map tile pyramid, with Principia-specific refinement logic layered on top.

9.1 Tile pyramid

Let the initial-condition domain be a fixed square $[0, 1]^2$. At zoom level z , the domain is divided into $2^z \times 2^z$ tiles. A tile is identified by (z, t_x, t_y) , and covers

$$u \in \left[\frac{t_x}{2^z}, \frac{t_x + 1}{2^z} \right], \quad v \in \left[\frac{t_y}{2^z}, \frac{t_y + 1}{2^z} \right].$$

Each tile has a fixed internal sample resolution $T \times T$.

9.2 Camera in world coordinates

A simple camera model is:

```
type Camera = {
    uCenter: number,
    vCenter: number,
    uHalf: number,
    vHalf: number,
```

```
screenW: number,
screenH: number
}
```

Panning changes the centre; zooming changes the half-widths.

9.3 Base zoom level

A useful screen-matching zoom estimate is

$$z_{\text{base}} = \text{clamp}\left(\left\lfloor \log_2\left(\frac{W}{T \Delta u_{\text{view}}}\right) \right\rfloor, 0, Z_{\text{max}}\right),$$

where W is the screen width in pixels, T is tile resolution, and Δu_{view} is the current viewport width in world coordinates.

This gives the level whose texel density most closely matches the current screen density.

9.4 Visible tile range

For a given level z , let $s = 2^z$. The tile range intersecting the viewport is

$$t_x^{\min} = \lfloor u_{\min} s \rfloor, \quad t_x^{\max} = \lfloor (u_{\max} - \varepsilon) s \rfloor,$$

and similarly for t_y , clamped to $[0, s - 1]$.

9.5 Ancestor fallback

When a tile at the desired level is not ready, the renderer should walk upward through parent tiles until it finds the nearest ready ancestor. That ancestor is then drawn upscaled over the missing child region.

This gives the key map-like behaviour:

- panning is always immediate,
- zooming never leaves large blank areas,
- detail sharpens progressively as finer tiles complete.

9.6 Render strategy: active tile quads

The simplest and most debuggable approach is to render a list of active tile quads rather than doing a full-screen per-fragment tree traversal.

Each active quad contains:

```
type ActiveRenderTile = {
  z: number,
  tx: number,
  ty: number,
  role: "ready" | "fallback",
  sourceTileId: number,
  uMin: number,
  uMax: number,
  vMin: number,
  vMax: number,
  textureSlot: number
}
```

A frame is then rendered as a set of non-overlapping quads that cover the viewport. Some quads correspond to actual ready leaf tiles; others correspond to fallback ancestors filling regions where children are still missing.

10 Quadtree scheduler and refinement

10.1 Tile lifecycle

The CPU-side quadtree manager should own the hierarchy, visibility tracking, queueing, cache eviction, and refinement logic. The GPU should focus on dense arithmetic: simulating tile payloads and rendering tile quads.

10.2 Pan and zoom behaviour

On camera changes:

1. compute the new visible tile set,
2. reuse cached tiles that still intersect the viewport,
3. enqueue newly visible missing tiles,
4. render visible regions immediately using ready tiles or nearest ready ancestors,
5. continue refining asynchronously in later frames.

No full cache flush should occur on ordinary panning.

10.3 Refinement priority

Normal slippy maps refine mainly by visibility and zoom match. Principia should extend this with dynamical complexity.

A useful schematic score is

$$P = w_v P_{\text{visible}} + w_z P_{\text{zoom}} + w_c P_{\text{complexity}} + w_f P_{\text{focus}},$$

where complexity can depend on coherence, impurity, ensemble spread, suspect fraction, or any other tile summary that signals unresolved structure.

This is the crucial difference between map-style tiling and scientific adaptive refinement: smooth regions can stay coarse, while chaotic boundaries can be refined more deeply even if visually small.

11 Precision strategy for deep zoom

To maintain precision at deep quadtree levels, tile centres and extents should be computed on the CPU in `f64`. For each tile, pass the centre and half-widths to the GPU, and let the shader compute tile-local sample coordinates relative to that centre.

A tile uniform block can therefore contain:

```
struct TileUniforms {
    cu: f64,
    hu: f64,
    cv: f64,
    hv: f64
}
```

The shader then reconstructs sample coordinates using local offsets. This avoids catastrophic precision loss when zooming deeply into the parameter domain.

12 Recommended implementation order

The best development order is to build the instrumentation and render substrate first, then add the quadtree and heavy compute once the tooling exists.

1. Render contract and shader infrastructure

- stable `RenderContext`,
- colour node,
- brightness node,
- fixed combiner,
- postprocess hook,
- hot reload,
- helper library.

2. Debug rendering

- sample debug modes,
- tile debug modes,
- quadtree overlays,
- cache and fallback visualisation.

3. Flat-grid compute backend

- write canonical sample payloads,
- feed them into the render graph,
- validate through the debug views.

4. Tiled dispatch without adaptive refinement

- visible tile range calculation,
- tile objects,
- tiled compute jobs,
- active quad rendering,
- ancestor fallback.

5. Adaptive quadtree scheduler

- split / keep / merge,
- priority queue,
- cache management,
- refinement by complexity.

This order makes failures intelligible. If the quadtree later misbehaves, the debug infrastructure is already present to answer whether the issue lies in the payload, the reduction, the lookup, the refinement score, or the cache.

13 Suggested project structure

```
src/  
  render/  
    renderGraph.ts  
    shaderRegistry.ts  
    shaderCompiler.ts  
    shaderHotReload.ts  
    uniforms.ts  
    renderState.ts  
    builtins/  
      colour/  
      brightness/  
      combiner/  
      post/  
  shaders/  
    shared/  
      types.wgsl  
      context.wgsl  
      colour_spaces.wgsl  
      transfer_functions.wgsl  
      shape_sphere.wgsl  
      vmf.wgsl  
      cvd.wgsl  
    templates/  
      fragment_template.wgsl  
    builtins/  
      colour_*.wgsl  
      brightness_*.wgsl  
      combiner_*.wgsl  
      post_*.wgsl  
  editor/  
    shaderEditorModel.ts  
    shaderDocs.ts  
    uniformEditor.ts
```

14 Summary

The proposed design can be summarised as follows.

1. Use one canonical diagnostics payload for rendering, debugging, refinement, and export.
2. Represent the renderer as a fixed graph: colour, brightness, combiner, postprocess.
3. Expose built-ins and user customisation through the same shader contracts and helper library.
4. Treat debug shaders and overlays as core infrastructure, not as temporary hacks.
5. Implement the viewport as a slippy-map style quadtree with ancestor fallback and active tile quads.
6. Add adaptive refinement only after the render and debug substrate is already solid.

This architecture keeps the scientific semantics clear, supports your own shader-centric workflow, gives advanced users the same machinery you rely on, and creates a robust foundation for

the later adaptive quadtree and compute pipeline.